```
policy:
  type: ACL
  criteria:
    - name: block_tcp
      classifier:
        network_src_port_id: 640dfd77-c92b-45a3-b8fc-22712de480e1
        destination_port_range: 80-1024
        ip_proto: 6
        ip_dst_prefix: 192.168.1.2/24
    - name: block_udp
      classifier:
        network_src_port_id: 640dfd77-c92b-45a3-b8fc-22712de480eda
        destination_port_range: 80-1024
        ip_proto: 17
        ip_dst_prefix: 192.168.2.2/24
```

Figure 10: VNF graph template

```
openstack vnf descriptor create --vnfd-file tosca-vnffg-vnfd1.yaml VNFD1
openstack vnf descriptor create --vnfd-file tosca-vnffg-vnfd2.yaml VNFD2

$ openstack vnf create --vnfd-name VNFD1 VNF1
$ openstack vnf create --vnfd-name VNFD2 VNF2
$ openstack vnf graph descriptor create --vnffgd-file \
    tosca-vnffgd-sample.yaml <NAME: tosca-vnffgd-sample>
$ openstack vnf graph create --vnffgd-name <VNFFGD: tosca-vnffgd-sample> \
   <NAME: tosca-vnffg-sample>
$ openstack vnf graph list

$ openstack vnf graph delete <VNFFG: tosca-vnffg-sample>
```

Figure 11: Deploying and deleting SFC

The VNFGD, or virtual network function graph descriptor, is a document or file that describes the dependencies and connections between various VNFs in a network. VNFs are software versions of network functions like load balancers, firewalls, and VPNs that can run on regular hardware. The VNFGD typically uses a standard format like OpenStack TOSCA, making it easy for VNF management tools to read and process.

In an NFV system, VNFGDs are used to automate VNF deployment and management by enabling administrators to declaratively describe the dependencies and relationships between VNFs. This facilitates the provisioning of resources and configuration of VNFs through automation. VNFGDs are an essential component of the NFV ecosystem as they provide a mechanism to specify VNF dependencies and interactions in a standardised manner.

By using VNFGDs, administrators can define and control VNFs with ease through automation tools, increasing network programmability and flexibility. Figure 10 shows a sample VNF graph template where two rules based on IP address are used to either block a TCP packet or a UDP packet.

Steps for SFC deployment in OpenStack

Step 1: First, onboard VNF templates from the Tosca template, give a name to them and then launch them.

Step 2: Now create a descriptor for the VNF forwarding graph, and then launch the graph.

***Step 3*:** List and show the VNF graph deployed.

Step 4: Delete the VNF graph deployed.

The commands to deploy the above-mentioned steps in OpenStack are shown in Figure 11.

We hope this article will be a valuable resource for those looking to understand the basics of VNF deployment on OpenStack, and how it can be used to create powerful and flexible network services using service function chaining. END

References

- Yamato, Yoji & Muroi, Masahito & Tanaka, Kentaro & Uchimura, Mitsutomo. (2014). Development of template management technology for easy deployment of virtual resources on OpenStack. Journal of Cloud Computing: Advances, Systems, and Applications. 3. 7. 10.1186/s13677-014-0007-3
- Sales, William & Coutinho, Emanuel & Souza, Jose. (2017). Auto-Scaling in NFV Using Tacker
- OpenStack SDN Skydiving into Service Function Chaining, *https://networkop.co.uk/blog/2017/09/15/os-sfc-skydive/*
- Software Defined Networking, *https://avinetworks.com/glossary/software-defined-networking/*
- Nithya Ganesan, Shubham Aggarwal and B. Thangaraju, 'Performance Analysis of SDN controllers within an OpenStack infrastructure', 2022 IEEE India Council International Subsections Conference (INDISCON), Bhubaneswar, India, 2022, pp. 1-7, *https://doi.org/10.1109/INDISCON54605.2022.9862876*
- Tacker architecture, *https://docs.openstack.org/tacker/wallaby/user/architecture.html*

By: Satya Jyoti Das, Astha Borkataky, Nithya Ganesan and Dr B. Thangaraju

The authors are associated with the Open Source Technology Lab at the International Institute of Information Technology, Bengaluru.